

Desktop Agent Integration for the Rademics ERP

A lightweight Windows companion app for authentication, attendance, and activity tracking — working alongside the existing ERP website

TARGET SYSTEM	HOSTING	DOCUMENT TYPE
rademics.52digit.com	Hostinger VPS · Cloudflare DNS	Architecture & Rollout Plan

1. Objective

The goal is to build a small **Windows desktop application** that works alongside the existing ERP website — not instead of it. The website stays the single place where employees, HR, and admins do their actual work: tasks, attendance history, reports, leave, and dashboards.

The desktop app has one narrow job: confirm who is sitting at the computer, know when they've checked in, and quietly track activity while they're checked in.

The desktop app is responsible for:

- User authentication (login)
- Check In / Check Out
- Tracking system activity while running
- Sending activity data to the ERP backend
- Displaying notifications (optional)

Hard requirement

The application must **never auto-start** with Windows. Tracking only begins after the employee manually opens the app and clicks **Check In**.

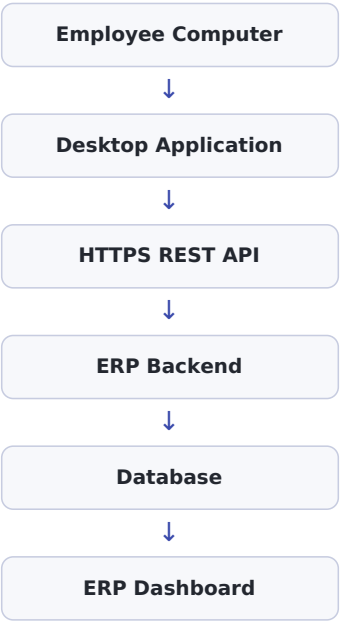
2. Existing System

Nothing about the current infrastructure needs to change. The desktop app is purely additive — it plugs into what's already running.

Component	Details
ERP Website	Existing web application — remains the primary interface for all business operations
Hosting	Hostinger VPS
DNS	Managed via Cloudflare
Domain	rademics.52digit.com

3. High-Level Architecture

The desktop app never touches the database directly. Everything flows through secure REST (or GraphQL) APIs, the same way the website does.



Backend Technology

The backend can be built on any stack the team is comfortable with — for example PHP/Laravel, ASP.NET Core, Node.js, Java Spring Boot, Python Django, Go, or Rust. The choice doesn't affect the desktop app, since it only ever talks to REST/GraphQL endpoints.

4. User Roles & Features

Employee	HR	Admin
- Login - Check In - Check Out - View assigned tasks - Update task status - View attendance - View notifications - View profile	- Employee management - Attendance monitoring - Leave approval - Task assignment - Attendance reports	- Full system access - Manage employees & HR users - Manage departments - Manage desktop app versions - View reports & live status

5. Desktop Application Scope

The desktop app is deliberately **not** a second ERP. It should never duplicate business functionality that already lives on the website. Its job stays narrow:

- Authenticate the user
- Start attendance
- Monitor activity while checked in
- Send activity data periodically
- Stop monitoring after Check Out

Application Modules

Module	Purpose
Login	Authenticates the employee and stores tokens securely
Check In	Starts the attendance session
Check Out	Ends the attendance session
Activity Monitor	Watches keyboard/mouse activity, idle time, lock/sleep state
API Client	Handles all communication with the backend
Notification Manager	Displays incoming notifications (optional)
Update Checker	Checks for and prompts/installs new versions
Settings	Local app configuration

6. Employee Workflow

This is the step-by-step lifecycle of a single work session, from opening the app to checking out.

1 Open the application

Employee opens the desktop app. Nothing is tracked yet — it stays idle until login.

2 Log in

The app calls `POST /api/auth/login`. The backend returns an access token, refresh token, user role, and employee details.

3 Check In

Attendance begins: the start time is recorded and desktop monitoring switches on.

4 Continuous activity tracking

Every 30–60 seconds, the app checks keyboard/mouse activity, idle duration, system lock status, and sleep status, then sends the result to `POST /api/activity`:

Field	Description
<code>employeeId</code>	Identifies the employee
<code>timestamp</code>	When the reading was taken
<code>status</code>	ACTIVE or IDLE
<code>idleSeconds</code>	How long the machine has been idle

5 Working normally

Status is **ACTIVE** whenever keyboard or mouse activity is detected, and switches to **IDLE** once there's no activity for a configured duration.

Business decision needed: app closed mid-session

If the employee closes the app while still checked in, the org must choose: **(A)** automatically check the employee out, or **(B)** keep the session active until they reopen the app.

6 Check Out

Attendance ends and monitoring stops. The backend stores check-in time, check-out time, total working time, active time, and idle time.

7. ERP Website Responsibilities

The website keeps handling everything that isn't check-in/check-out or live activity tracking:

- Dashboard
- Attendance history
- Tasks
- Reports
- Leave
- Payroll
- Employee profiles
- Notifications

No business functionality should be duplicated in the desktop app unless there's a specific reason to.

8. Backend APIs

Authentication

Endpoint	Purpose
POST /api/auth/login	Authenticate the user, issue tokens
POST /api/auth/logout	End the session, invalidate tokens
POST /api/auth/refresh	Refresh an expired access token

Attendance

Endpoint	Purpose
POST /api/checkin	Start an attendance session
POST /api/checkout	End an attendance session
GET /api/today-attendance	Fetch today's attendance status

Activity

Endpoint	Purpose
POST /api/activity	Submit a periodic activity reading
GET /api/activity/status	Fetch current activity status

Tasks

Endpoint	Purpose
GET /api/tasks	List assigned tasks
PUT /api/tasks/{id}	Update a task's status

Notifications, Profile & Updates

Endpoint	Purpose
GET /api/notifications	Fetch notifications
GET /api/profile	Fetch the employee's profile
GET /api/desktop/version	Check the latest desktop app version
GET /api/desktop/download	Download the installer

9. Database Tables

Suggested tables to support the above — grouped by area:

Area	Tables
People & Access	Users, Employees, Roles, Departments
Attendance	Attendance, Attendance Logs, Desktop Sessions
Activity	Activity Logs
Work	Tasks, Task Assignments
System	Notifications, Devices, Application Versions

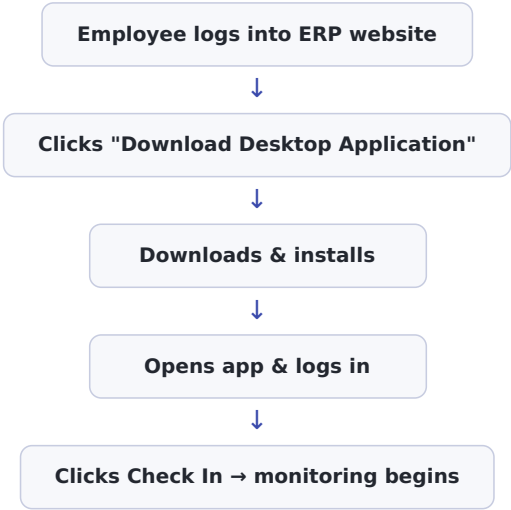
10. Installer & Employee Installation

Building & Hosting the Installer

Build the desktop app into a single installer, `ERP-Agent-Setup.exe` , and host it on the existing VPS:

```
https://rademics.52digit.com/downloads/ERP-Agent-Setup.exe
```

Installation Flow



Update Flow (optional)

The app checks `GET /api/desktop/version` . If a newer version exists, it either prompts the user to update or downloads the installer automatically.

11. Security

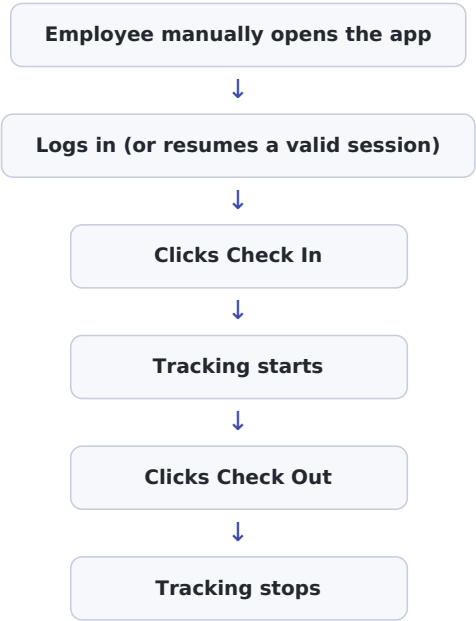
- HTTPS only for all communication
- JWT or OAuth2 access tokens, with refresh tokens for session renewal
- Role-based authorization on every endpoint
- Encrypt tokens stored on the local machine
- Validate every API request server-side
- Log activity data securely
- Never expose database credentials to the desktop application

12. Suggested Development Order

- PHASE 1** Review the existing ERP; identify reusable authentication and attendance logic.
- PHASE 2** Build REST APIs for authentication, attendance, activity, tasks, and notifications.
- PHASE 3** Develop the Windows desktop application; implement login and secure API communication.
- PHASE 4** Add Check In / Check Out; start and stop monitoring based on those actions.
- PHASE 5** Add system activity detection while the application is running.
- PHASE 6** Create an installer and host it on the existing VPS.
- PHASE 7** Test with one employee, then HR, then Admin.
- PHASE 8** Roll out to all employees.

13. Key Functional Requirement — No Auto-Start

This is the one rule that shapes the entire app: it must never launch automatically when Windows starts, and it must never track anything the employee hasn't explicitly opted into for that session.



Closing the app mid-session

This should follow whichever attendance policy the organization decides — for example, prompting the user to check out first, or automatically ending the session on close.